



25. 貪婪演算法

2026資訊研究社算法班
Made with ❤️ by jheanlee



什麼是貪婪演算法

貪婪演算法(greedy algorithm) 是一種演算法的類別，此類演算法希望在每一步的當下選擇最佳作法。簡單來說，就是不顧未來最終結果、也不顧過去的判斷，而是選擇當下的最佳解(locally optimal solution)。

貪婪演算法也可以被視為特殊狀況的動態規劃(dynamic programming)。

注意：因為貪婪演算法只顧當下的最佳解，他不能保證得到最佳解。

知名的貪婪演算法有：Dijkstra演算法(尋路)、A* 搜尋演算法(尋路)、Kruskal演算法(最小生成樹)、赫夫曼編碼(壓縮)



知名例子 - 找零問題

在一間商店, 客人用了大鈔付錢, 而店員計算後共要找回 h 元。有 m 種紙鈔/硬幣可以使用, 且面額為 $a[i]$ 。保證 $a[i] > a[i - 1]$, when $i > 0$ 。

請問怎麼找零給的硬幣數量最少? 輸出硬幣數量及使用的硬幣。

評分方式:

若最佳解分數為 k , 較最佳解硬幣數量少 a 則得 $k - a$ 分。超時則0分。

範例1: 範例2:

5 4 6 3

1 2 3 4 1 3 4

貪婪不一定是最佳解

在範例二可以發現，使用貪婪解並不是最佳解：

貪婪解：

3

4 1 1

最佳解：

2

3 3

這題可以用動態規劃保證最佳解，但貪婪比動態規劃更快。

DP 時間複雜度: $O(n*m)$ 空間複雜度: $O(n+m)$

貪婪 時間複雜度: $O(m)$ 空間複雜度: $O(m + \text{ans})$

如果題目評分寬鬆且數值範圍極大，使用貪婪演算法是更好的解。

```
int main() {
    int n, m;
    cin >> n >> m;
    vector<int> coins;

    for (int i = 0; i < m; i++) {
        int tmp;
        cin >> tmp;
        coins.push_back(tmp);
    }
}
```

```
int p = m - 1;
vector<int> used;
while (n > 0 && p >= 0) {
    while (p >= 0 && n < coins[p]) {
        p--;
    }

    if (p < 0) {
        cout << -1 << '\n';
        return 0;
    }

    for (int i = 0; i < n / coins[p]; i++) {
        used.push_back(coins[p]);
    }

    n %= coins[p];
}
```

```
cout << used.size() << '\n';
for (int i = 0; i < used.size() - 1; i++) {
    cout << used[i] << ' ';
}
cout << used[used.size() - 1] << '\n';

return 0;
}
```



Leetcode 948 - Bag of Tokens

你有初始能量 $power$ 、初始分數 0 與一包代幣 $tokens$ 。你可以使用任意數量的代幣(不用全用) 並取得最多分數。

針對每個未使用過的代幣可以做兩種操作：

1. 使用代幣 i 、消耗 $tokens[i]$ 的能量, 並取得 1 分。(使用前的能量必需不小於 $tokens[i]$)
2. 使用代幣 i 、取得 $tokens[i]$ 的能量, 並失去 1 分。



Leetcode 948 - Bag of Tokens

```
class Solution {
public:
    int bagOfTokensScore(vector<int>& tokens, int power) {
        sort(tokens.begin(), tokens.end());

        int p = 0, q = tokens.size() - 1;
        int score = 0;

        while (p <= q) {
            if (power >= tokens[p]) {
                power -= tokens[p];
                score++;
                p++;
            } else if (q - p > 1 && score > 0) {
                power += tokens[q];
                score--;
                q--;
            } else {
                break;
            }
        }

        return score;
    }
};
```